

IF5153 Natural Language Processing

Laporan Praktikum 2
Text Classification

Disusun untuk memenuhi tugas mata kuliah Pemrosesan Bahasa Alami
pada Semester 1 (satu) Tahun Akademik 2025/2026
Tugas Besar IF5153 Pemrosesan Bahasa Alami



Oleh

Shabrina Maharani

13522134

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
BANDUNG
2025

I. Deskripsi Persoalan

Text classification adalah salah satu task dalam bidang Natural Language Processing (NLP) yang bertujuan untuk mengelompokkan teks ke dalam kategori tertentu berdasarkan isi atau maknanya. Salah satu penerapannya adalah analisis sentimen, yaitu mengidentifikasi apakah suatu teks mengandung opini positif, negatif, atau netral.

Pada praktikum kali ini, mahasiswa diminta untuk menganalisis bagaimana membangun sistem klasifikasi teks untuk analisis sentimen dengan menggunakan metode shallow machine learning. Tahapan yang dilakukan meliputi preprocessing data, feature extraction menggunakan metode seperti Bag of Words, TF-IDF, dan Embedding, serta penerapan beberapa algoritma klasifikasi seperti Naive Bayes, k-Nearest Neighbor (kNN), Support Vector Machine (SVM), dan beberapa algoritma lainnya. Tujuan akhirnya adalah membandingkan performa dari berbagai kombinasi metode ekstraksi fitur dan algoritma klasifikasi untuk menentukan pendekatan yang paling efektif dalam menghasilkan model analisis sentimen.

II. Analisis Preprocessing

Tahap preprocessing bertujuan untuk mengubah teks mentah menjadi bentuk yang lebih terstruktur sehingga bisa diolah oleh algoritma machine learning. Selain itu, tahapan ini juga berfungsi untuk mengurangi noise atau anomali yang terdapat pada teks mentah agar representasi data menjadi lebih konsisten. Pada tahap ini, bisa juga dilakukan customization, misalnya, kata-kata tertentu yang memiliki makna spesifik atau secara langsung mencerminkan suatu label sentimen dapat dipertahankan atau diproses dengan aturan khusus supaya informasi penting tersebut tidak hilang.

1. **Lowercasing** untuk mengubah seluruh teks menjadi huruf kecil untuk menyeragamkan bentuk kata menghindari saat tokenization di-generate kata yang duplikat hanya karena perbedaan kapital.
2. **Normalisasi slang/akronim** dengan mengganti kata informal atau singkatan dengan bentuk baku sesuai kamus.

3. **Tokenisasi** dengan memecah kalimat menjadi unit kata atau token, jadi memudahkan analisis.
4. **Spelling correction** dengan memperbaiki kesalahan ketik menggunakan *spellchecker* agar variasi kata tidak terlalu berlebihan.
5. **Negation Handling** untuk menggabungkan kata negasi dengan kata sesudahnya (misalnya *not good* → *not_good*) untuk menjaga konteks semantik.
6. **Stopword removal** dilakukan untuk menghapus kata-kata umum yang tidak memiliki kontribusi besar terhadap informasi sentimen. Namun, pada eksperimen ini proses tersebut dikustomisasi dengan tetap mempertahankan kata-kata tertentu seperti *not*, *but*, *very*, dan *however*. Alasan utamanya adalah karena kata-kata tersebut sering berfungsi sebagai penguat makna semantik sentimen atau sebagai penanda adanya pergeseran opini. Contohnya, pada kalimat “good but slow”, klausa setelah *but* biasanya menjadi bagian yang lebih menentukan makna keseluruhan, sehingga keberadaan kata *but* penting untuk dipertahankan.
7. **Lemmatization** yaitu mengembalikan kata ke bentuk dasarnya untuk menyatukan variasi morfologi (misalnya *running*, *ran* → *run*).

Pada eksperimen ini, saya melakukan perbandingan antara preprocessing dengan dan tanpa *stopword removal*. Misalnya, pada kombinasi TF-IDF unigram dengan model yang sama, hasil Macro F1 mencapai 0.8329 ketika menggunakan *stopword removal*, sedangkan tanpa *stopword removal* nilainya hanya 0.7568. Perbedaan ini menunjukkan bahwa *stopword removal* bisa meningkatkan performa model.

Alasan utamanya adalah karena *stopword* yang sangat sering muncul cenderung menambah *noise* dalam representasi data dan mengaburkan kata-kata yang benar-benar penting untuk menentukan polaritas sentimen. Dengan menghapus *stopword*, distribusi bobot pada fitur menjadi lebih terfokus pada kata-kata bermakna sentimen, sehingga model lebih mudah mengenali pola. Meski demikian, kata-kata tertentu seperti *not*, *but*, atau *very* tetap dipertahankan karena berperan sebagai pengubah atau penguat sentimen. Oleh karena itu, strategi *stopword removal* yang dikustomisasi ini menjadi langkah yang efektif dalam menjaga keseimbangan antara pengurangan *noise* dan pelestarian sinyal semantik penting.

III. Analisis Feature Extraction

Setelah teks melalui tahap preprocessing, langkah berikutnya adalah mengubah teks menjadi representasi numerik agar bisa diproses oleh algoritma *machine learning*. Dalam eksperimen ini digunakan beberapa metode ekstraksi fitur dengan karakteristik yang berbeda:

1. **Bag of Words (BoW)**

BoW merepresentasikan dokumen sebagai vektor frekuensi kemunculan kata, tanpa memperhatikan urutan kata. Misalnya, kalimat “*the movie is good*” dan “*good movie the is*” akan memiliki representasi yang sama karena hanya menghitung jumlah kata.

2. **TF-IDF (Term Frequency–Inverse Document Frequency)**

TF-IDF tidak cuma mempertimbangkan frekuensi kemunculan kata tetapi juga mempertimbangkan seberapa jarang kata tersebut muncul di seluruh korpus. Kata yang umum seperti *the* atau *is* akan mendapat bobot kecil, sedangkan kata yang jarang muncul seperti *excellent* atau *boring* akan mendapat bobot lebih tinggi. Ada dua metode yang saya gunakan dalam eksperimen yaitu:

- **Unigram**: hanya memperhitungkan kata tunggal sebagai fitur.
- **Bigram**: memperhitungkan pasangan kata yang berurutan, sehingga dapat menangkap konteks sederhana seperti “*not good*”.

3. **SVD (Singular Value Decomposition)**

SVD digunakan sebagai teknik reduksi dimensi pada BoW dan TF-IDF. Dengan SVD, dimensi fitur yang sangat tinggi dapat diringkas menjadi representasi berdimensi rendah (misalnya 200), sehingga lebih efisien untuk diproses. Tapi tetap ada risiko informasi penting ikut hilang.

4. **Word Embedding (Word2Vec, SBERT)**

- **Word2Vec**: memetakan kata ke dense vector sehingga kata-kata dengan makna serupa punya representasi yang berdekatan. Dalam eksperimen ini, representasi dokumen dihitung dengan merata-ratakan vektor kata dalam dokumen.
- **SBERT (Sentence-BERT)**: menghasilkan vektor kalimat kontekstual menggunakan model transformer, sehingga dapat menangkap makna lebih kompleks dibandingkan Word2Vec.

Dari hasil eksperimen ditunjukkan bahwa **TF-IDF Unigram dan BoW** justru memberikan performa tertinggi dengan nilai *Macro F1* mencapai **0.8329**, dibandingkan dengan metode embedding. Berikut adalah analisis faktor yang bisa mempengaruhi hasil tersebut:

1. Ukuran Dataset

Dataset yang digunakan pada eksperimen kali ini relatif kecil (586 sampel latih, 146 sampel uji). Pada kondisi ini, kehadiran keyword dengan bobot sentimen yang besar (misalnya *excellent*, *awful*, *boring*) lebih dominan, jadi representasi sederhana berbasis frekuensi seperti BoW dan TF-IDF sudah cukup efektif.

2. Karakteristik Word2Vec

Implementasi Word2Vec dalam eksperimen ini menggunakan rata-rata vektor kata dalam satu dokumen. Pendekatan ini cenderung "meratakan" sinyal, sehingga kata kunci sentimen bisa kehilangan power dari maknanya ketika dikombinasikan dengan kata lain yang netral.

3. Keterbatasan SBERT

Meskipun SBERT menghasilkan vektor kontekstual yang lebih kompleks dan kaya akan pemahaman kata, model ini umumnya membutuhkan dataset yang lebih besar atau kasus klasifikasi yang lebih kompleks agar lebih efektif. Pada dataset kecil dengan tugas seperti klasifikasi sentimen, representasi berbasis frekuensi masih lebih baik dalam menghasilkan model yang efektif.

4. Pengaruh SVD

Penerapan *dimensionality reduction* dengan TruncatedSVD pada BoW dan TF-IDF tidak selalu menghasilkan peningkatan skor. Hal ini biasanya dapat disebabkan oleh hilangnya informasi penting saat jumlah dimensi fitur direduksi, terutama ketika dataset relatif kecil.

Dari sisi perbandingan antar fitur, terlihat bahwa BoW dan TF-IDF unigram menghasilkan distribusi skor yang lebih stabil di berbagai algoritma *shallow machine learning* dibandingkan Word2Vec maupun SBERT. Hal ini menunjukkan bahwa metode sederhana berbasis frekuensi kata masih sangat bagus terutama untuk dataset kecil dengan label sentimen yang cukup eksplisit.

IV. Analisis Model Shallow ML

Pada eksperimen ini diuji delapan algoritma *shallow machine learning* untuk melihat efektivitasnya dalam klasifikasi sentimen teks. Masing-masing algoritma memiliki mekanisme yang berbeda dalam memproses data teks yang telah direpresentasikan dalam bentuk vektor (misalnya hasil BoW atau TF-IDF).

1. Naive Bayes (Multinomial & Gaussian)

Naive Bayes bekerja berdasarkan Teorema Bayes dengan asumsi bahwa setiap fitur (kata) bersifat independen satu sama lain. Pada teks, Multinomial Naive Bayes menghitung probabilitas kata muncul dalam suatu kelas (positif/negatif) dan memilih kelas dengan probabilitas terbesar. Kelebihannya sederhana, cepat, dan sering menjadi baseline yang baik untuk teks. Tapi, asumsi bahwa setiap kata berdiri sendiri tanpa saling berhubungan sering tidak sesuai dengan kenyataan, sehingga akurasinya jadi terbatas.

Sementara itu, Gaussian Naive Bayes digunakan ketika fitur yang dianalisis bersifat kontinu. Model ini mengasumsikan bahwa nilai fitur dalam setiap kelas mengikuti distribusi normal (*Gaussian distribution*). Dalam konteks teks, Gaussian Naive Bayes lebih cocok diterapkan pada representasi vektor berdimensi rendah dan kontinu, seperti hasil reduksi dimensi dengan SVD atau word embeddings. Akan tetapi, karena distribusi nilai fitur pada data teks tidak selalu Gaussian, performanya biasanya lebih rendah dibandingkan Multinomial Naive Bayes yang memang lebih sesuai untuk data diskrit seperti BoW atau TF-IDF.

2. K-Nearest Neighbor (KNN)

KNN mengklasifikasikan sebuah dokumen berdasarkan kedekatannya dengan dokumen lain di ruang fitur. Jarak biasanya diukur dengan *cosine similarity* atau *Euclidean distance*. Semakin banyak tetangga terdekat (k), semakin stabil prediksi kelas. Kekurangannya, KNN tidak efisien pada data berdimensi tinggi seperti teks, karena perhitungan jarak menjadi mahal dan kadang tidak informatif.

3. **Support Vector Machine (SVM) & LinearSVC**

SVM bekerja dengan mencari *hyperplane* yang memisahkan kelas dengan margin terbesar. Pada data teks, yang cenderung *sparse* dan berdimensi tinggi, SVM sangat efektif karena fokus pada *support vectors* (dokumen yang paling dekat dengan batas klasifikasi). LinearSVC adalah versi optimisasi berbasis *linear kernel* yang lebih efisien untuk dataset besar.

4. **Random Forest**

Random Forest adalah *ensemble* dari banyak pohon keputusan. Setiap pohon dilatih pada subset acak dari data dan fitur, lalu hasilnya digabungkan melalui voting mayoritas. Keunggulannya terletak pada kemampuannya menangkap interaksi non-linear antar fitur dan ketahanannya terhadap *overfitting*.

5. **XGBoost (Extreme Gradient Boosting)**

XGBoost adalah model *boosting* berbasis pohon keputusan. Alih-alih membangun pohon secara paralel seperti Random Forest, XGBoost membangun pohon secara bertahap dengan memfokuskan pembelajaran pada kesalahan dari pohon sebelumnya. Model ini terkenal sangat kuat, efisien, dan biasanya menghasilkan performa tinggi pada berbagai kompetisi *machine learning*.

Berdasarkan tabel hasil, model **SVM** dan **Random Forest** secara konsisten memberikan performa terbaik.

- **SVM** unggul karena sangat cocok untuk data teks yang direpresentasikan oleh TF-IDF. Kemampuannya dalam mencari *hyperplane* dengan margin maksimal membuat SVM bisa mengklasifikasikan teks dengan baik walaupun ruang fiturnya berdimensi sangat tinggi.
- **Random Forest** dan **XGBoost** juga menunjukkan performa kuat dengan adanya mekanisme *ensemble* yang bisa menangkap hubungan non-linear antar kata. Random Forest relatif lebih stabil, sedangkan XGBoost lebih baik dalam memperbaiki kesalahan.
- Selain itu, **Naive Bayes** dan **KNN** menghasilkan performa yang lebih rendah. Dalam teks, asumsi bahwa kata-kata bersifat independen pada Naive Bayes sering tidak sesuai kenyataan, sedangkan KNN kurang efektif karena perhitungan jarak di ruang fitur yang sangat besar (ribuan kata) tidak selalu merepresentasikan kemiripan makna antar dokumen.

Secara keseluruhan, hasil ini menegaskan bahwa algoritma berbasis margin (SVM) dan *ensemble* berbasis pohon (Random Forest, XGBoost) lebih unggul dalam klasifikasi teks dibandingkan metode probabilistik sederhana (Naive Bayes) atau metode berbasis jarak (KNN).

V. Kesimpulan

Berdasarkan rangkaian eksperimen yang telah dilakukan, dapat disimpulkan bahwa kombinasi tahap pre-processing yang tepat, metode ekstraksi fitur sederhana berbasis frekuensi kata, serta pemilihan model klasifikasi yang sesuai sangat berpengaruh terhadap performa analisis sentimen. Hasil menunjukkan bahwa TF-IDF unigram dan BoW memberikan hasil terbaik, dengan performa tertinggi dicapai oleh model SVM berkat kemampuannya menangani data teks yang berdimensi tinggi dan sparse. Selain itu, strategi stopwords removal yang dikustomisasi terbukti efektif dalam menjaga makna semantik penting, khususnya kata-kata negasi, intensitas (very, dan lain-lain), dan kontras (but). Sementara itu, metode berbasis embedding dan algoritma probabilistik sederhana seperti Naive Bayes atau metode berbasis jarak seperti KNN cenderung kurang kompetitif pada dataset kecil. Dengan demikian, dapat disimpulkan bahwa untuk kasus

analisis sentimen, pendekatan shallow machine learning yang dipadukan dengan representasi fitur sederhana masih menjadi pilihan yang efektif dan efisien.

VI. Lampiran Hasil Tabel Perbandingan Nilai Macro F1 Score

index	Preprocessing	Feature	Model	Macro F1
10	with_stopwords	TFIDF_unigram	SVM	0.8328874901652242
4	with_stopwords	BoW	Random Forest	0.8328874901652242
6	with_stopwords	BoW	XGBoost	0.8192401960784313
11	with_stopwords	TFIDF_unigram	Random Forest	0.7973901098901099
12	with_stopwords	TFIDF_unigram	LinearSVC	0.7957513768686073
55	no_stopwords	BoW	LinearSVC	0.7916610077414098
3	with_stopwords	BoW	SVM	0.7916610077414098
5	with_stopwords	BoW	LinearSVC	0.7916610077414098
83	no_stopwords	BoW	SVM	0.7705555555555557
56	no_stopwords	SBERT	Gaussian Naive Bayes	0.7696212417024599
60	no_stopwords	TFIDF_unigram	SVM	0.7568681318681318
13	with_stopwords	TFIDF_unigram	XGBoost	0.7490696438064859
36	with_stopwords	SBERT	Gaussian Naive Bayes	0.7484088528784648
75	no_stopwords	BoW_SVD	SVM	0.7469362745098039
18	with_stopwords	TFIDF_bigram	SVM	0.7406593406593407
58	no_stopwords	BoW	Multinomial Naive Bayes	0.7281587807903598
20	with_stopwords	TFIDF_bigram	LinearSVC	0.7281587807903598
71	no_stopwords	TFIDF_bigram	XGBoost	0.7275567824310743
57	no_stopwords	BoW_SVD	LinearSVC	0.7251819219569016
26	no_stopwords	BoW	XGBoost	0.7214791502753737
7	with_stopwords	BoW_SVD	LinearSVC	0.7214791502753737
0	with_stopwords	BoW	Multinomial Naive Bayes	0.7214791502753737
25	with_stopwords	BoW_SVD	SVM	0.7163461538461539
61	no_stopwords	TFIDF_unigram	Random Forest	0.7091943127962086
54	no_stopwords	BoW	Random Forest	0.7091943127962086

Show 25 per page

1234

index	Preprocessing	Feature	Model	Macro F1
78	no_stopwords	BoW_SVD	XGBoost	0.7051474262868566
21	with_stopwords	TFIDF_bigram	XGBoost	0.7038087962457711
34	with_stopwords	TFIDF_bigram_SVD	LinearSVC	0.6974358974358974
63	no_stopwords	TFIDF_unigram	XGBoost	0.6907396622015143
62	no_stopwords	TFIDF_unigram	LinearSVC	0.6863370547581074
91	no_stopwords	SBERT	LinearSVC	0.6863370547581074
73	no_stopwords	BoW_SVD	Logistic Regression	0.675314465408805
70	no_stopwords	TFIDF_bigram	LinearSVC	0.6644549763033176
23	with_stopwords	BoW_SVD	Logistic Regression	0.6644549763033176
68	no_stopwords	TFIDF_bigram	SVM	0.6644549763033176
1	with_stopwords	BoW	Logistic Regression	0.6542124542124542
51	no_stopwords	BoW	Logistic Regression	0.6542124542124542
17	with_stopwords	TFIDF_bigram	KNN	0.630761775872827
19	with_stopwords	TFIDF_bigram	Random Forest	0.6289308176100629
84	no_stopwords	TFIDF_bigram_SVD	LinearSVC	0.6289308176100629
42	with_stopwords	SBERT	XGBoost	0.6289308176100629
28	with_stopwords	BoW_SVD	XGBoost	0.610989010989011
65	no_stopwords	TFIDF_bigram	Gaussian Naive Bayes	0.6026936026936027
15	with_stopwords	TFIDF_bigram	Gaussian Naive Bayes	0.6026936026936027
9	with_stopwords	TFIDF_unigram	KNN	0.5989804587935429
69	no_stopwords	TFIDF_bigram	Random Forest	0.5905286793223107
38	with_stopwords	SBERT	KNN	0.5905286793223107
41	with_stopwords	SBERT	LinearSVC	0.5825471698113207
89	no_stopwords	SBERT	SVM	0.5825471698113207
93	no_stopwords	Word2Vec	Gaussian Naive Bayes	0.5809659090909091

index	Preprocessing	Feature	Model	Macro F1
37	with_stopwords	SBERT	Logistic Regression	0.555703211517165
31	with_stopwords	TFIDF_bigram_SVD	KNN	0.555703211517165
52	no_stopwords	BoW	KNN	0.5544078014184397
35	with_stopwords	TFIDF_bigram_SVD	XGBoost	0.5488530161427357
39	with_stopwords	SBERT	SVM	0.5488530161427357
97	no_stopwords	Word2Vec	Random Forest	0.5423556827719944
2	with_stopwords	BoW	KNN	0.5352437865000679
99	no_stopwords	Word2Vec	XGBoost	0.5302369660246445
49	with_stopwords	Word2Vec	XGBoost	0.5245421245421246
47	with_stopwords	Word2Vec	Random Forest	0.5245421245421246
74	no_stopwords	BoW_SVD	KNN	0.5137362637362637
43	with_stopwords	Word2Vec	Gaussian Naive Bayes	0.5125645438898451
90	no_stopwords	SBERT	Random Forest	0.5083333333333333
85	no_stopwords	TFIDF_bigram_SVD	XGBoost	0.5083333333333333
83	no_stopwords	TFIDF_bigram_SVD	Random Forest	0.5083333333333333
32	with_stopwords	TFIDF_bigram_SVD	SVM	0.5083333333333333
81	no_stopwords	TFIDF_bigram_SVD	KNN	0.5083333333333333
59	no_stopwords	TFIDF_unigram	KNN	0.5083333333333333
40	with_stopwords	SBERT	Random Forest	0.5083333333333333
7	with_stopwords	TFIDF_unigram	Multinomial Naive Bayes	0.5083333333333333
76	no_stopwords	BoW_SVD	Random Forest	0.5063333333333333
33	with_stopwords	TFIDF_bigram_SVD	Random Forest	0.5063333333333333
26	with_stopwords	BoW_SVD	Random Forest	0.5063333333333333
14	with_stopwords	TFIDF_bigram	Multinomial Naive Bayes	0.5063333333333333
92	no_stopwords	SBERT	XGBoost	0.5034330011074197

76 to 100 of 100 entries					
Index	Preprocessing	Feature	Model		Macro F1
88	no_stopwords	SBERT	KNN		0.5034330011074197
67	no_stopwords	TFIDF_bigram	KNN		0.5034330011074197
87	no_stopwords	SBERT	Logistic Regression		0.5034330011074197
79	no_stopwords	TFIDF_bigram_SVD	Gaussian Naive Bayes		0.49424768779607486
30	with_stopwords	TFIDF_bigram_SVD	Logistic Regression		0.45622119815668205
94	no_stopwords	Word2Vec	Logistic Regression		0.45622119815668205
95	no_stopwords	Word2Vec	KNN		0.45622119815668205
96	no_stopwords	Word2Vec	SVM		0.45622119815668205
24	with_stopwords	BoW_SVD	KNN		0.45622119815668205
98	no_stopwords	Word2Vec	LinearSVC		0.45622119815668205
58	no_stopwords	TFIDF_unigram	Logistic Regression		0.45622119815668205
82	no_stopwords	TFIDF_bigram_SVD	SVM		0.45622119815668205
80	no_stopwords	TFIDF_bigram_SVD	Logistic Regression		0.45622119815668205
8	with_stopwords	TFIDF_unigram	Logistic Regression		0.45622119815668205
44	with_stopwords	Word2Vec	Logistic Regression		0.45622119815668205
45	with_stopwords	Word2Vec	KNN		0.45622119815668205
46	with_stopwords	Word2Vec	SVM		0.45622119815668205
66	no_stopwords	TFIDF_bigram	Logistic Regression		0.45622119815668205
48	with_stopwords	Word2Vec	LinearSVC		0.45622119815668205
64	no_stopwords	TFIDF_bigram	Multinomial Naive Bayes		0.45622119815668205
16	with_stopwords	TFIDF_bigram	Logistic Regression		0.45622119815668205
57	no_stopwords	TFIDF_unigram	Multinomial Naive Bayes		0.45622119815668205
29	with_stopwords	TFIDF_bigram_SVD	Gaussian Naive Bayes		0.31351001939237233
72	no_stopwords	BoW_SVD	Gaussian Naive Bayes		0.23376623376623376
22	with_stopwords	BoW_SVD	Gaussian Naive Bayes		0.22430109080401647